

Database Normalization Activity

Construction Project Dataset — 1NF, 2NF, 3NF, BCNF, 4NF

Group: 12

Members: Clarisse Mukayiranga, Erica Karekezi Ishimwe, Gloire Gwiza, Jongkuch Isaac Chol Anyar, Norette Atete

Source file: big3_construction_raw_data.csv (15 data rows)

Step 1: Understanding the Raw Data

The raw file stores one row per (Project, Worker) pairing taken from a project's crew. Because every project has several workers, all project-level, client-level, supervisor-level and supplier-level facts are repeated on every row for that project, and several cells hold more than one value separated by "|".

Entities identified

- Project — ProjectID, ProjectName, ProjectType, StartDate, EndDate, SiteAddress, SiteCity, SiteState
- Client — ClientName, ClientPhone, ClientEmail, ClientCity
- Supervisor — SupervisorName, SupervisorPhone
- Worker — WorkerName, WorkerPhone, WorkerHourlyRate, plus Skills and Certifications used on a project
- Supplier — SupplierName, SupplierCity, SupplierPhones
- Material — MaterialSupplied, MaterialUnitCost (supplied by a Supplier to a Project)
- Equipment — EquipmentUsed, EquipmentRentalCost (rented by a Project)

Columns repeated across many rows

ProjectName, ProjectType, StartDate, EndDate, SiteAddress/City/State, ClientName/Phone/Email/City, SupervisorName/Phone, SupplierCity, and SupplierPhones are repeated on every worker row that belongs to the same project or the same supplier. This is the redundancy that 2NF and 3NF remove.

Columns containing multiple values in one cell (separated by |)

- WorkerSkills, WorkerCertifications — multiple skills/certifications per worker-on-project
- SupplierPhones — multiple phone numbers per supplier
- MaterialSupplied / MaterialUnitCost — a paired multi-valued list (material + its cost)
- EquipmentUsed / EquipmentRentalCost — a paired multi-valued list (equipment + its cost)

Candidate identifiers

- ProjectID identifies a Project
- WorkerName identifies a Worker (no WorkerID was supplied, so the name is used as the natural key)
- ClientName, SupervisorName, SupplierName identify those entities (no surrogate IDs were supplied)
- The combination (ProjectID, WorkerName) identifies one crew assignment row in the raw file

Dependencies and relationships discovered by inspecting the data

Before decomposing, we verified the following functional dependencies (FDs) directly against the CSV values (not just assumed from real-world intuition):

- $\text{ProjectID} \rightarrow \text{ProjectName}, \text{ProjectType}, \text{StartDate}, \text{EndDate}, \text{SiteAddress}, \text{SiteCity}, \text{SiteState}, \text{ClientName}, \text{SupervisorName}$ (one-to-many: $\text{Project} \rightarrow \text{Worker}$)
- $\text{ClientName} \rightarrow \text{ClientPhone}, \text{ClientEmail}, \text{ClientCity}$
- $\text{SupervisorName} \rightarrow \text{SupervisorPhone}$
- $\text{WorkerName} \rightarrow \text{WorkerPhone}, \text{WorkerHourlyRate}$ (confirmed constant for each worker across every project they appear in)
- $\text{SupplierName} \rightarrow \text{SupplierCity}$ (confirmed constant for each supplier)
- $(\text{ProjectID}, \text{WorkerName}) \rightarrow \text{SupplierName}$ — IMPORTANT: a worker's supplier is NOT fixed; the same worker uses a different supplier on different projects, and two workers on the SAME project can use different suppliers (e.g. Project P001 uses both BuildPro Supplies and SteelWorks Inc). The supplier therefore depends on the whole (Project, Worker) pair, not on either column alone.
- $(\text{ProjectID}, \text{SupplierName}, \text{Material}) \rightarrow \text{UnitCost}$ — the same material (e.g. "Glass Panels" in Project P004) can be supplied by two different suppliers in the same project at the same cost, so SupplierName must be part of the key, not a column determined by it.
- $(\text{ProjectID}, \text{Equipment}) \rightarrow \text{RentalCost}$ — equipment rental cost is set per project, independent of which worker or supplier the row happens to mention.
- $(\text{ProjectID}, \text{WorkerName}) \twoheadrightarrow \text{Skill}$ and $(\text{ProjectID}, \text{WorkerName}) \twoheadrightarrow \text{Certification}$ — these are two INDEPENDENT multi-valued facts about a crew assignment (a worker applies a particular subset of skills/certifications on a given project; the sets are not the same across projects for the same worker, and the skill list is unrelated to the certification list).
- $\text{SupplierName} \twoheadrightarrow \text{Phone}$ — a supplier can have several independent phone numbers.

$\text{Project} \leftrightarrow \text{Worker}$ is many-to-many (one project has many workers, one worker appears on many projects). $\text{Worker} \leftrightarrow \text{Skill}$ and $\text{Worker} \leftrightarrow \text{Certification}$ are one(assignment)-to-many. $\text{Project} \leftrightarrow \text{Supplier}$ (via Material) is many-to-many. $\text{Supplier} \leftrightarrow \text{Phone}$ is one-to-many.

Stage 1: First Normal Form (1NF)

Files: 1NF/project_worker_core_1nf.csv, worker_skills_1nf.csv, worker_certifications_1nf.csv, supplier_phones_1nf.csv, materials_1nf.csv, equipment_1nf.csv

Which columns violated 1NF?

WorkerSkills, WorkerCertifications, SupplierPhones, MaterialSupplied, MaterialUnitCost, EquipmentUsed and EquipmentRentalCost all held multiple "|" -separated values in a single cell.

How did we make the values atomic?

Each multi-valued cell was exploded into one row per value. Paired lists (Material+UnitCost and Equipment+RentalCost) were split by matching list position, so each material keeps its own cost and each piece of equipment keeps its own rental cost.

Did we create new rows, new tables, or both?

Both. New rows were created for every exploded value, and because each multi-valued attribute is a logically different list, each one was placed in its own new table (worker_skills_1nf, worker_certifications_1nf, supplier_phones_1nf, materials_1nf, equipment_1nf), separate from the single-valued columns that stayed in project_worker_core_1nf.

What key identifies each row?

- project_worker_core_1nf.csv: (ProjectID, WorkerName) — mirrors the grain of the original raw row
- worker_skills_1nf.csv: (ProjectID, WorkerName, Skill)
- worker_certifications_1nf.csv: (ProjectID, WorkerName, Certification)
- supplier_phones_1nf.csv: (ProjectID, WorkerName, Phone)
- materials_1nf.csv: (ProjectID, WorkerName, Material)
- equipment_1nf.csv: (ProjectID, WorkerName, Equipment)

At this stage we deliberately kept (ProjectID, WorkerName) as the key everywhere, exactly as it appears in the source row, because 1NF only requires atomic values and a unique row key — it does not yet ask us to analyse which columns truly depend on which part of that key. That analysis is the job of 2NF.

Note on redundancy at this stage (by design)

1NF intentionally still contains heavy redundancy: project_worker_core_1nf.csv repeats every project's name, dates, site address, client details and supervisor details once for every worker on that project (e.g. all three workers on Project P001 each carry a full copy of P001's data, and the client "Metro Corp" details are copied wherever that client's projects appear). This is expected — 1NF's only job is atomicity and row uniqueness. We do not remove this redundancy yet because doing so requires analysing functional dependencies, which is the explicit purpose of 2NF (partial dependencies) and 3NF (transitive dependencies), covered next.

Stage 2: Second Normal Form (2NF)

Files: 2NF/project_2nf.csv, worker_2nf.csv, project_worker_supplier_2nf.csv, worker_skills_2nf.csv, worker_certifications_2nf.csv, supplier_phones_2nf.csv, materials_2nf.csv, equipment_2nf.csv

Which tables had composite keys?

All six 1NF tables used the composite key (ProjectID, WorkerName) or an extension of it.

Which columns depended only on part of a composite key?

- In project_worker_core_1nf: ProjectName, ProjectType, StartDate, EndDate, SiteAddress, SiteCity, SiteState, ClientName, ClientPhone, ClientEmail, ClientCity, SupervisorName, SupervisorPhone depend only on ProjectID (verified: identical for every worker row of the same project).
- In project_worker_core_1nf: WorkerPhone and WorkerHourlyRate depend only on WorkerName (verified: identical for every project the same worker appears on).
- In materials_1nf (key ProjectID, WorkerName, Material): UnitCost does not depend on WorkerName at all — two different workers who use the same supplier on the same project list identical materials/costs. It depends on (ProjectID, SupplierName, Material).
- In equipment_1nf (key ProjectID, WorkerName, Equipment): RentalCost does not depend on WorkerName — it is fixed per (ProjectID, Equipment) regardless of which worker's row mentions it.

By contrast, SupplierName in project_worker_core_1nf, and Skill/Certification in their tables, depend on the **WHOLE** key (ProjectID, WorkerName) — neither ProjectID alone nor WorkerName alone determines them (a worker uses different

suppliers/skills on different projects, and different workers on the same project can use different suppliers). These are correctly NOT partial dependencies, so they stay as-is.

Which data did we move into separate tables?

- Project-only columns → project_2nf.csv (PK ProjectID)
- Worker-only columns → worker_2nf.csv (PK WorkerName)
- SupplierName (+ SupplierCity, temporarily) stays attached to the (ProjectID, WorkerName) pair → project_worker_supplier_2nf.csv
- WorkerName dropped from materials — materials_2nf.csv is now keyed on (ProjectID, SupplierName, Material)
- WorkerName dropped from equipment — equipment_2nf.csv is now keyed on (ProjectID, Equipment)

Primary keys / foreign keys introduced

- project_2nf.csv: PK ProjectID
- worker_2nf.csv: PK WorkerName
- project_worker_supplier_2nf.csv: PK (ProjectID, WorkerName); FK ProjectID → project_2nf, FK WorkerName → worker_2nf
- worker_skills_2nf.csv / worker_certifications_2nf.csv: PK (ProjectID, WorkerName, Skill/Certification); FK (ProjectID, WorkerName) → project_worker_supplier_2nf
- materials_2nf.csv: PK (ProjectID, SupplierName, Material)
- equipment_2nf.csv: PK (ProjectID, Equipment); FK ProjectID → project_2nf

Why is the design now in 2NF?

Every table is in 1NF, and every non-key column now depends on the ENTIRE key of its table rather than on a subset of it. The partial dependencies on ProjectID alone or WorkerName alone (for project facts, worker facts, material cost and equipment cost) have all been removed.

Stage 3: Third Normal Form (3NF)

Files: 3NF/client_3nf.csv, supervisor_3nf.csv, project_3nf.csv, supplier_3nf.csv, project_worker_supplier_3nf.csv, supplier_phones_3nf.csv, worker_3nf.csv, worker_skills_3nf.csv, worker_certifications_3nf.csv, materials_3nf.csv, equipment_3nf.csv

Which transitive dependencies did we find?

- In project_2nf: ProjectID → ClientName → ClientPhone, ClientEmail, ClientCity. ClientName is a non-key column that determines other non-key columns — a transitive dependency.
- In project_2nf: ProjectID → SupervisorName → SupervisorPhone — same pattern.
- In project_worker_supplier_2nf: (ProjectID, WorkerName) → SupplierName → SupplierCity — SupplierCity depends on the non-key column SupplierName, not on the key directly.
- In supplier_phones_2nf: (ProjectID, WorkerName) → SupplierName → Phone — the phone numbers belong to the supplier, not to the project/worker pair that happened to mention that supplier.

Which non-key attributes depended on other non-key attributes?

ClientPhone/Email/City depended on ClientName; SupervisorPhone depended on SupervisorName; SupplierCity and the supplier's phone numbers depended on SupplierName — in every case a non-key attribute determined by another non-key attribute, rather than by the table's actual key.

New tables created

- client_3nf.csv (PK ClientName) — split out of project
- supervisor_3nf.csv (PK SupervisorName) — split out of project
- supplier_3nf.csv (PK SupplierName) — split out of project_worker_supplier
- supplier_phones_3nf.csv (PK SupplierName, Phone) — ProjectID/WorkerName dropped since phones belong to the supplier alone

Foreign keys used to preserve relationships

- project_3nf.csv keeps ClientName and SupervisorName as foreign keys into client_3nf and supervisor_3nf
- project_worker_supplier_3nf.csv keeps SupplierName as a foreign key into supplier_3nf (SupplierCity removed)
- materials_3nf.csv keeps SupplierName as a foreign key into supplier_3nf

Why is the design now in 3NF?

Every table is in 2NF, and no non-key column depends on another non-key column anymore: ClientName, SupervisorName and SupplierName are still present as foreign keys (so the relationships are preserved), but the facts that used to hang transitively off them (phone numbers, email, city) now live in the table where they are determined directly by the key.

Stage 4: Boyce-Codd Normal Form (BCNF)

Files: BCNF/ — same eleven tables as 3NF (client, supervisor, project, supplier, project_worker_supplier, supplier_phones, worker, worker_skills, worker_certifications, materials, equipment)

Functional dependencies checked (formal check)

For every 3NF table we listed every functional dependency $X \rightarrow Y$, identified all candidate keys of that table, and confirmed whether X is one of them:

Table	Functional Dependency ($X \rightarrow Y$)	Determinant (X)	Candidate Key(s)	X is Candidate Key?
project_3nf	ProjectID $\rightarrow$ ProjectName, ProjectType, StartDate, EndDate, SiteAddress, SiteCity, SiteState, ClientName, SupervisorName	ProjectID	ProjectID	Yes
client_3nf	ClientName $\rightarrow$ ClientPhone, ClientEmail, ClientCity	ClientName	ClientName	Yes
supervisor_3nf	SupervisorName $\rightarrow$ SupervisorPhone	SupervisorName	SupervisorName	Yes
supplier_3nf	SupplierName $\rightarrow$ SupplierCity	SupplierName	SupplierName	Yes
worker_3nf	WorkerName $\rightarrow$ WorkerPhone, WorkerHourlyRate	WorkerName	WorkerName	Yes
project_worker_supplier_3nf	(ProjectID, WorkerName) $\rightarrow$ SupplierName	(ProjectID, WorkerName)	(ProjectID, WorkerName)	Yes
materials_3nf	(ProjectID, SupplierName, Material) $\rightarrow$ UnitCost	(ProjectID, SupplierName, Material)	(ProjectID, SupplierName, Material)	Yes
equipment_3nf	(ProjectID, Equipment) $\rightarrow$ RentalCost	(ProjectID, Equipment)	(ProjectID, Equipment)	Yes
supplier_phones_3nf	no non-key attribute (trivial)	(SupplierName, Phone)	(SupplierName, Phone)	Yes (trivial)
worker_skills_3nf	no non-key attribute (trivial)	(ProjectID, WorkerName, Skill)	(ProjectID, WorkerName, Skill)	Yes (trivial)

Table	Functional Dependency ($X \rightarrow Y$)	Determinant (X)	Candidate Key(s)	X is Candidate Key?
worker_certifications_3nf	no non-key attribute (trivial)	(ProjectID, WorkerName, Certification)	(ProjectID, WorkerName, Certification)	Yes (trivial)

Did we find any determinant that was not a candidate key?

No. The table above shows that in all eleven 3NF tables, every determinant we identified is already the full candidate key of its table (or the table has no non-key attribute at all, satisfying BCNF trivially).

Which tables violated BCNF?

None. This is an expected and explicitly allowed outcome — the assignment brief notes that some tables may already satisfy BCNF once they reach 3NF. Our design reached that point because the earlier 2NF/3NF decomposition was driven by FDs verified directly against the data (e.g. discovering that SupplierName depends on the full (ProjectID, WorkerName) pair, and that material cost requires SupplierName inside the key) rather than by guessing a "typical" schema.

How did we decompose the table(s)?

No decomposition was required at this stage.

Candidate keys after this stage

Unchanged from 3NF: ProjectID; ClientName; SupervisorName; SupplierName; WorkerName; (ProjectID, WorkerName); (ProjectID, SupplierName, Material); (ProjectID, Equipment); (SupplierName, Phone); (ProjectID, WorkerName, Skill); (ProjectID, WorkerName, Certification).

Why is the design now in BCNF?

For every functional dependency $X \rightarrow Y$ across all eleven tables, X is a candidate key of its table, which is exactly the BCNF requirement.

Stage 5: Fourth Normal Form (4NF)

Files: 4NF/ — same eleven tables as BCNF

Multi-valued dependencies identified

- (ProjectID, WorkerName) $\twoheadrightarrow$ Skill and (ProjectID, WorkerName) $\twoheadrightarrow$ Certification: a crew assignment has an independent list of skills used and an independent list of certifications used. Knowing the skills tells you nothing about the certifications and vice versa.
- ProjectID $\twoheadrightarrow$ Material (via Supplier) and ProjectID $\twoheadrightarrow$ Equipment: a project independently has a list of materials supplied and a list of equipment rented. These two lists are unrelated to each other.
- SupplierName $\twoheadrightarrow$ Phone: a supplier independently has a list of phone numbers.

Which independent lists were mixed together (in the raw data)?

In the original flat file, WorkerSkills and WorkerCertifications were two unrelated multi-valued lists crammed into the same row, and MaterialSupplied/MaterialUnitCost and EquipmentUsed/EquipmentRentalCost were two unrelated multi-valued lists also crammed into the same row. Storing two independent multi-valued facts in one table forces a meaningless cross-product between them (e.g. pairing every skill with every certification) and is exactly the situation 4NF forbids.

How did we separate them?

We never recombined them: worker_skills and worker_certifications have been two separate tables since 1NF, and materials and equipment have been two separate tables since 1NF as well. Carrying that separation through 2NF/3NF/BCNF means no table in our final design mixes two independent multi-valued facts, so no further decomposition is required at this stage.

Primary keys of the relationship tables

- worker_skills_4nf.csv: (ProjectID, WorkerName, Skill)
- worker_certifications_4nf.csv: (ProjectID, WorkerName, Certification)
- materials_4nf.csv: (ProjectID, SupplierName, Material)
- equipment_4nf.csv: (ProjectID, Equipment)
- supplier_phones_4nf.csv: (SupplierName, Phone)

Why is the final design in 4NF?

The design is already in BCNF, and every multi-valued dependency we identified has its own dedicated table whose key is exactly the set of columns that participate in that multi-valued fact. No table contains two or more independent multi-valued attributes, so there are no non-trivial multi-valued dependencies left to resolve.

Final Schema Summary

- Project(ProjectID, ProjectName, ProjectType, StartDate, EndDate, SiteAddress, SiteCity, SiteState, ClientName FK, SupervisorName FK)
- Client(ClientName, ClientPhone, ClientEmail, ClientCity)
- Supervisor(SupervisorName, SupervisorPhone)
- Worker(WorkerName, WorkerPhone, WorkerHourlyRate)
- ProjectWorkerSupplier(ProjectID FK, WorkerName FK, SupplierName FK)
- WorkerSkill(ProjectID FK, WorkerName FK, Skill)
- WorkerCertification(ProjectID FK, WorkerName FK, Certification)
- Supplier(SupplierName, SupplierCity)
- SupplierPhone(SupplierName FK, Phone)
- Material(ProjectID FK, SupplierName FK, Material, UnitCost)
- Equipment(ProjectID FK, Equipment, RentalCost)

Entity-Relationship Diagram

Final Entity-Relationship Diagram — Construction Project Database

